CS 300 Project One

Ilianna Chavez

Pseudocode

```
STRUCT Course: courseNumber, courseTitle, prerequisites
VECTOR loadCourses(fileName)
create empty vector; open file; if open fails, print error and
return
for each line: split by comma; if fewer than 2 values, print
error and return
create Course from first two values; add remaining values as
prerequisites; add Course to vector
for each Course and prerequisite: search vector for matching
courseNumber; if missing, print error
return vector
VECTOR printAllCourses(courses): sort by courseNumber lowest to
highest; print courseNumber and title
VECTOR searchCourse(courses, courseNumber): scan courses; when
found, print course and each prerequisite
HASH loadCourses(fileName, table)
open file; if open fails, print error and return; create list
of course numbers
first pass: split each line; validate at least 2 values; save
course numbers
```

second pass: create Course; validate each prerequisite against course-number list; hash courseNumber; insert

HASH printAllCourses(table): copy all courses to list; sort list by courseNumber; print list

HASH searchCourse(table, courseNumber): hash key; search table; print course and prerequisite course information

BST loadCourses(fileName)

create empty tree; open file; if open fails, print error and return

for each line: split by comma; validate at least 2 values; create Course; add prerequisites; insert by courseNumber

after loading, validate that each prerequisite matches a courseNumber in the tree; close file

BST insertCourse(node, course): if empty create node; if smaller go left; otherwise go right

BST printAllCourses(node): if node exists, visit left; print courseNumber and title; visit right

BST searchCourse(tree, courseNumber): start at root; compare key; move left or right until found; print course/prereqs

MAIN MENU

repeat until choice = 9

print "1. Load Data Structure 2. Print Course List 3. Print Course 9. Exit "

if 1: ask for file name and load courses

```
if 2: print all courses in alphanumeric order
if 3: ask for course number and search/print course information
if 9: exit; otherwise print "Invalid option"
```

Runtime Analysis

For the file-loading portion, n represents the number of courses. The vector reads and creates each course in linear time, but the prerequisite validation in my milestone scans the vector again, so the worst case is $O(n^2)$. The hash table uses two passes through the file; insertion is normally fast, but in the worst case collisions can make each insertion linear, giving $O(n^2)$. The binary search tree can also reach $O(n^2)$ during loading if the tree becomes unbalanced and each new course travels down a long chain.

Structure	Main load work	Worst-case total	Why
Vector	Read/create n courses + prerequisite validation	$O(n^2)$	Nested prerequisite/course scan can compare many courses.
Hash Table	Two file passes + hash insertions	$O(n^2)$	Worst-case collisions can make an insertion O(n).
Binary Search Tree	Read/create n courses + tree insertions	$O(n^2)$	An unbalanced tree can make each insertion O(n).

Advantages, Disadvantages, and Recommendation

A vector is straightforward to use and works well when courses are loaded and processed in sequence. The downside is that searching for one course is O(n), and the courses must be sorted before option 2 can print them in order. A hash table is strong for looking up one course because a good hash function can make searches very fast on average. Its weakness for this project is that a hash table does not naturally keep the courses in alphanumeric order, so I would still need to copy or collect the courses and sort them before printing the full list. A binary search tree fits both main requirements well because course numbers determine where each course is stored. Searching can be O(log n) when the tree is reasonably balanced, and an in-order traversal prints the courses from lowest to highest without a separate sorting step. Its main disadvantage is that an unbalanced tree can fall to O(n) search and $O(n^2)$ loading in the worst case. I would choose the binary search tree for ABCU because the program needs both individual course searches and an alphanumeric course list. It gives a clean way to support both operations, while still matching the tree logic I used in my milestone.